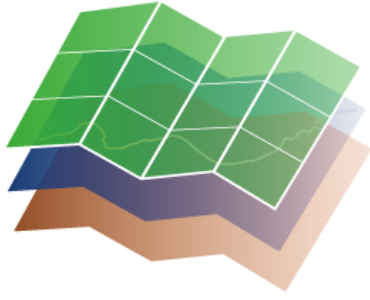




MAPA

CIUDADANO

Sprites

Script para la generación de sprites

Contenido

Versiones	3
1. Definición de sprite	4
2. Generación de sprites	4
3. Archivos de salida	6
4. Incluir sprite en JSON de estilos	8
5. Cargar Sprites en Mapbox GL	10

Versiones

Versión	Fecha	Comentarios
1.0	28/11/2024	Creación y actualización

1. Definición de sprite

Un sprite es un conjunto de imágenes bidimensionales combinadas en una sola, que se encuentran ordenadas en una matriz, de tal forma que cada imagen ocupa una posición dentro de dicha matriz. Un sprite permite representar símbolos gráficos personalizados en un mapa que ayudan a los usuarios a interpretar la información del mapa de forma más fácil y mejorar el tiempo de carga y rendimiento del mapa.

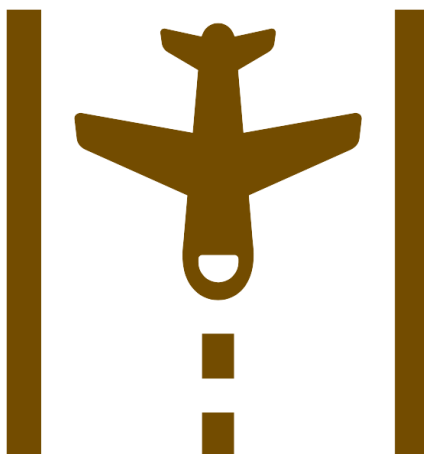
2. Generación de sprites

Para generar sprites hay que seguir los siguientes pasos:

1. Definir los símbolos que se quieren representar en el mapa.
2. Obtener y/o crear las imágenes individuales que van a formar la imagen combinada. Este paso se puede realizar de dos formas:
 - Diseñando y creando las imágenes a partir de cualquier herramienta de diseño gráfico, cómo por ejemplo, *Inkscape*, que permite dibujar, cambiar el color, modificar el tamaño, exportarlo en diferentes formatos como png, svg (*Gráficos Vectoriales Escalables*), etc.
 - Descargándolo desde algún servicio publicado.
3. Una vez que se obtienen todas las imágenes individuales, se incluyen en una misma carpeta.

A continuación, se muestra un ejemplo de una imagen individual obtenida:

Se ha generado un icono individual para representar en el mapa los aeropuertos y aeródromos. Este símbolo se ha exportado en formato svg y se va a utilizar para la clase 'aerodromo' del elemento geográfico 'aerodromo_pto' del modelo del mapa base vector publicado en <https://ideespain.github.io/mapabase/>.



aerodromo_pto-simbolo-aerodromo.svg

4. Configuración del script de python para generar los sprites.

El script que se va a utilizar para generar sprites se denomina **'svg2png.py'**. Si abrimos el script, lo primero que aparecen son las librerías necesarias para generar los sprite que se incluyen mediante *'import'*.

Este script genera una imagen combinada a partir de la carpeta que contiene la lista de imágenes individuales (en este caso en formato svg) y obtiene como salida dos archivos, una imagen combinada en formato png y un archivo JSON. El usuario puede modificar este script ajustándolo a sus necesidades, por ejemplo, puede modificar la línea 16 del código, si el formato de las imágenes individuales que ha obtenido son diferentes al establecido en el script (*'svg'*). También puede modificar la altura (*target_height*) y anchura (*target_width*) de las imágenes, modificando el tamaño en las líneas 25 y 26 del código, que en dicho script se establece en 50, para ambos valores. Se muestra a continuación una parte del script.

```

1 import os
2 import json
3 import argparse
4 import cairosvg
5 from PIL import Image
6 import re
7 import xml.etree.ElementTree as ET
8
9 def preprocess_svg(svg_content):
10     # Replace any 'stroke-opacity' values that are not valid floats with a default value (e.g., 1.0)
11     svg_content = re.sub(r'stroke-opacity="param\(outline-opacity\) "', 'stroke-opacity="1.0"', svg_content)
12     return svg_content
13
14 def generate_combined_image_and_json(svg_folder, output_combined_png, output_json):
15     # List all SVG files in the specified folder
16     svg_files = [f for f in os.listdir(svg_folder) if f.endswith('.svg')]
17     svg_files = [os.path.join(svg_folder, f) for f in svg_files]
18
19     total_svgs = len(svg_files)
20     max_svgs_per_row = int(total_svgs**0.5) # Use the square root for a balanced grid
21     num_rows = (total_svgs + max_svgs_per_row - 1) // max_svgs_per_row
22     num_cols = (total_svgs + num_rows - 1) // num_rows
23
24     target_width = 50
25     target_height = 50
26
27     total_width = num_cols * target_width
28     total_height = num_rows * target_height
29
30     sprite_data = {}

```

5. Lanzamiento del script utilizando Python 3 en Ubuntu

Para lanzar el script de generación de sprites en el sistema operativo Ubuntu, hay que tener instalado en primer lugar Python 3.

Una vez instalado Python 3, el lanzamiento del script se va a realizar desde la terminal de comandos. Para ello, accedemos a la ruta dónde se ha guardado el script **'svg2png.py'** y la carpeta que contiene la lista de svg y abrimos un terminal. A continuación, escribimos en la terminal `python3`, que abre el intérprete interactivo de Python 3, lo que permite escribir y ejecutar códigos Python directamente desde la línea de comandos. A continuación, escribimos el siguiente código:

```
python3 svg2png.py ./iconos_test/svg_folder sprite.png sprite.json
```

Donde **'svg2png.py'** es el nombre del script de Python que va a generar la imagen combinada con una resolución de 50x50 píxeles a partir de las imágenes individuales, **'./iconos_test/svg_folder'** es la ruta de la carpeta que contiene la lista de los svg (modificar con la ruta del usuario), **'sprite.png'** y **'sprite.json'** son los nombres de los dos archivos de salida que se obtienen al lanzar el script.

3. Archivos de salida

Al lanzar el script para generar los sprites se obtienen dos archivos de salida, una imagen combinada en formato png denominada *sprite.png* con una resolución de 50x50 píxeles y un archivo JSON denominado *sprite.json*.

1. JSON del sprite

El archivo JSON del sprite contiene la información correspondiente sobre la posición y el tamaño de cada sprite individual dentro de la imagen combinada.

La estructura del JSON contiene los sprites, es decir, las diferentes imágenes, dónde cada uno describe un sprite individual con las siguientes propiedades:

- **nombre:** Nombre único para identificar el sprite individual.
- **height:** Altura de la imagen
- **pixelRatio:** Relación de aspecto del píxel
- **width:** Anchura de la imagen
- **x:** Posición x de la imagen dentro de la matriz
- **y:** posición y de la imagen dentro de la matriz

A continuación, se muestra un ejemplo de un archivo JSON de sprite:

```
1 {
2   "aerodromo_pto-simbolo-aerodromo": {
3     "height": 50,
4     "pixelRatio": 1,
5     "width": 50,
6     "x": 0,
7     "y": 0
8   },
9   "aerodromo_pto-simbolo-hidroaerodromo": {
10    "height": 50,
11    "pixelRatio": 1,
12    "width": 50,
13    "x": 50,
14    "y": 0
15  },
16  "carretera_autonomica_2do_nivel_lin-simbolo": {
17    "height": 50,
18    "pixelRatio": 1,
19    "width": 50,
20    "x": 100,
21    "y": 0
22  },
23  "servicio_instalacion_pto-simbolo-parque_tecnologias": {
24    "height": 50,
25    "pixelRatio": 1,
26    "width": 50,
27    "x": 150,
28    "y": 0
29  },
30  "aerodromo_pto-simbolo-aerodromo_helipuerto": {
```

sprite.json

2. Imagen combinada

El script genera un segundo archivo de salida, denominado '*sprite.png*' que contiene una combinación de todos los sprites individuales en una sola imagen. Estos sprites se encuentran organizados en una matriz, dónde cada celda contiene uno de los sprites individuales originales con una dimensión de 50x50 píxeles. A continuación, se muestra un ejemplo de una imagen de sprite.



sprite.png

4. Incluir sprite en JSON de estilos

Una vez generado el sprite combinado (*sprite.png*) y el JSON de descripción (*sprite.json*), se suben ambos archivos a un servidor público (se puede usar GitHub) y se obtienen las URLs de ambos archivos.

Para incluir el sprite en el JSON de estilos se van a seguir la especificaciones de maplibre <https://maplibre.org/maplibre-style-spec/sprite/>.

Se necesita la URL desde la que se puede cargar el sprite y se incluye en la propiedad "sprite" en una hoja de estilo MapLibre. A continuación, se muestra un ejemplo.

```
20 "sprite": "http://localhost:6543/sprites/sprite",
```

Al cargar un sprite, se puede hacer referencia a las imágenes del sprite en otras partes del json de estilos. Se puede crear una capa de símbolos con la propiedad "icon-image". Siguiendo con el ejemplo anterior, sería

```
"icon-image": "aerodromo_pto-simbolo-aerodromo",
```

En un JSON de estilos cada imagen contenida dentro de un sprite tienen un identificador, por tanto, lo primero que debe aparecer es el identificador de la imagen del sprite y posteriormente sus propiedades. A continuación, se muestra un ejemplo y la explicación de sus propiedades:

```
{
  "id": "aerodromo_pto-simbolo-aerodromo",
  "type": "symbol",
  "metadata": {"maputnik:comment": "10-22"},
  "source": "mapaBaseXYZ",
  "source-layer": "aerodromo_pto",
  "minzoom": 10,
  "filter": ["all", ["==", "clase", "aerodromo"]],
  "layout": {
    "icon-size": 0.45,
    "visibility": "visible",
    "icon-image": "aerodromo_pto-simbolo-aerodromo",
    "text-anchor": "center",
    "text-field": "{nombre}",
    "text-font": ["Noto Sans Regular"],
    "text-size": 7.5,
    "text-keep-upright": true,
    "text-transform": "uppercase",
    "text-offset": [0, -1],
    "text-justify": "left",
    "text-variable-anchor": ["bottom-right"],
    "icon-pitch-alignment": "auto",
    "text-max-angle": 45,
    "icon-rotate": 180
  },
  "paint": {
    "icon-opacity": 1,
    "text-halo-width": 1,
    "text-color": "rgba( 88, 58, 0, 1.00 )",
    "text-halo-color": "rgba(255, 255, 255, 1)"
  },
  "maxZoom": 22
},
```

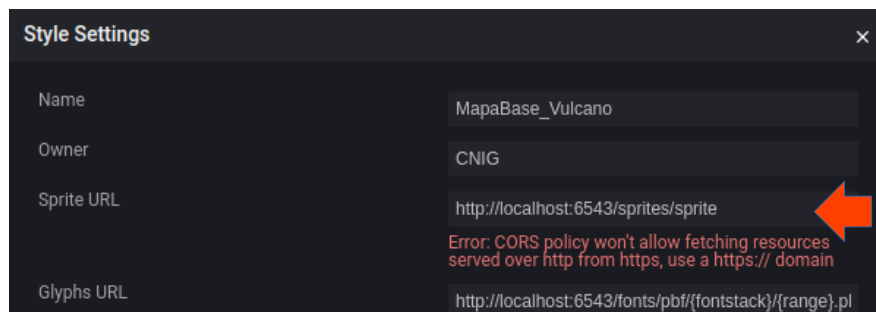
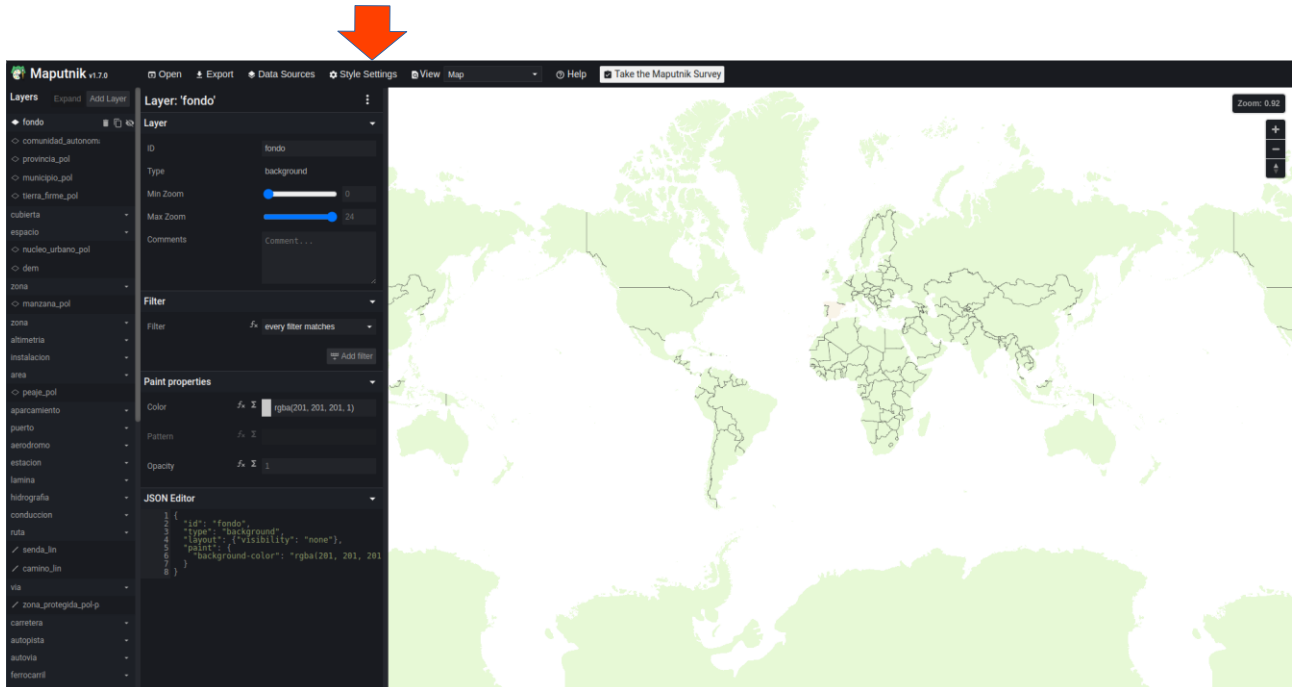
Donde:

- **id:** Identificador único de la capa.
- **type:** Se refiere al tipo de capa, en este caso se trata de un símbolo.

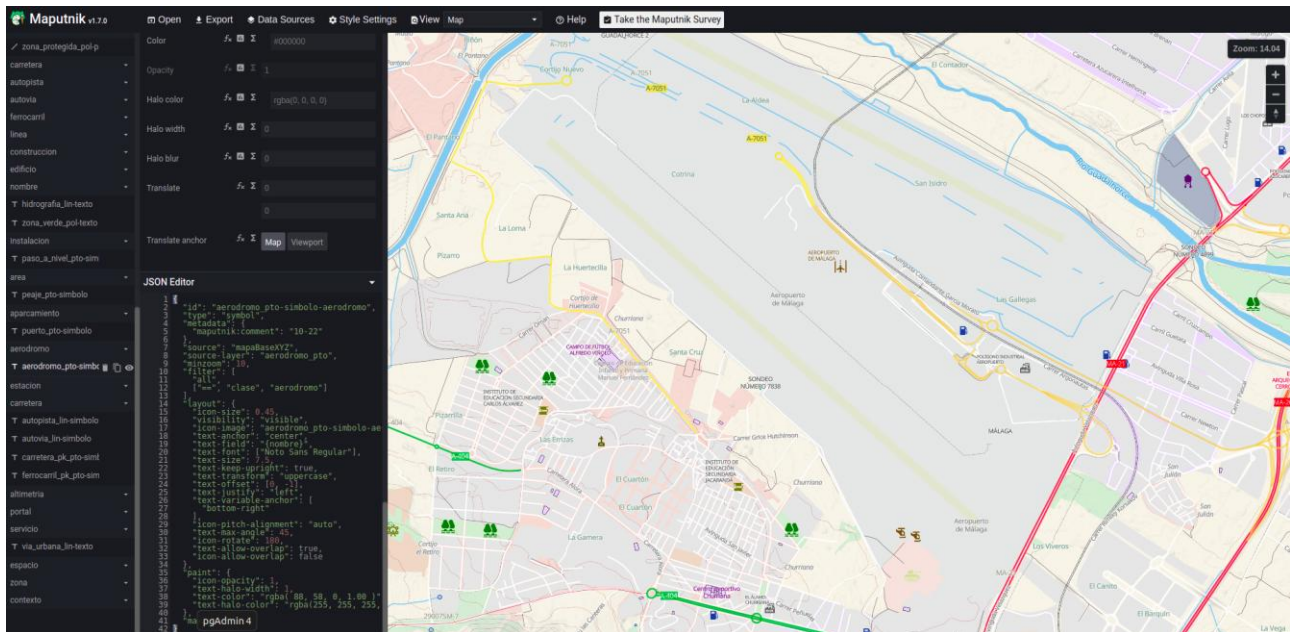
- **metadata:** Muestra información adicional, en este caso es un comentario de los niveles en los que aparecería el símbolo.
- **source:** Fuente de datos utilizada.
- **source-layer:** Capa específica utilizada dentro de la fuente de datos.
- **minzoom:** Nivel de zoom mínimo en el que la capa será visible.
- **filter:** Filtros que tienen que cumplirse para que la capa sea visible. En el ejemplo, se muestran los registros cuya clase es igual a 'aerodromo'.
- **layout:**
 - **icon-size:** Tamaño del símbolo en relación a su tamaño original.
 - **visibility:** Muestra la visibilidad de la capa. Los valores que toma son "visible" o "none".
 - **icon-image:** Nombre de la imagen que se va a utilizar desde el sprite.
 - **text-anchor:** Es la posición del texto con respecto al símbolo. En este caso "center".
 - **text-field:** Campo que se va a utilizar para mostrar el texto. En este caso se va a utilizar el campo "nombre".
 - **text-font:** Se refiere a la fuente utilizada para el texto.
 - **text-size:** Tamaño del texto.
 - **text-keep-upright:** Esta opción permite mantener el texto en posición vertical.
 - **text-transform:** Transforma el texto a mayúsculas:
 - Si se especifica "none", el texto no se modifica.
 - Si se especifica "uppercase", el texto se muestra en mayúsculas.
 - Si se especifica "lowercase", el texto se muestra en minúsculas.
 - **text-offset:** Desplaza el texto en relación al símbolo.
 - **text-justify:** Justificación del texto. Puede tomar los siguientes valores:
 - auto: El texto se alinea hacia la posición del ancla.
 - left: El texto se alinea a la izquierda.
 - center: El texto se centra.
 - right: El texto se alinea a la derecha.
 - **text-variable-anchor:** Puntos de anclaje variables para el texto que se utiliza para evitar superposiciones.
 - **icon-pitch-alignment:** Es la alineación del símbolo con respecto a la inclinación del mapa.
 - **text-max-angle:** Es el ángulo máximo entre las líneas adyacentes en un texto de varias líneas.
 - **icon-rotate:** Muestra en grados la rotación del símbolo.
- **paint:**
 - **icon-opacity:** Opacidad del símbolo.
 - **text-halo-width:** Tamaño del contorno del texto.
 - **text-color:** Color del texto asociado al símbolo.
 - **text-halo-color:** Color del contorno del texto.
- **maxZoom:** Nivel de zoom máxima en el que la capa será visible.

5. Cargar Sprites en Mapbox GL

A continuación, se va a cargar el sprite en una herramienta de edición de estilos que utiliza Mapbox GL, como por ejemplo, Maputnik. Para cargar el sprite en Maputnik, hay que copiar la URL del sprite combinado en '*Sprite URL*' que se encuentra dentro de la pestaña '*Style Settings*'. Una vez cargada la URL del sprite ya se puede usar los sprites en la edición de estilos.



Carga de sprites en Maputnik



Sprites cargados en Maputnik